



Universidade de Brasília

Departamento de Ciência da Computação

RISC-V Multiciclo Unidade Operativa



RISC-V Multiciclo

- Ideia: cada fase de execução de uma instrução dura um ciclo de relógio
- Instruções podem ser executadas em um número diferente de ciclos
- Ex.:
 - lw demora 5 ciclos
 - jump demora 3 ciclos
 - add demora 4 ciclos



RISC-V Multiciclo

- Ciclo dimensionado de acordo com a **etapa** mais demorada
- Vamos reutilizar unidades funcionais
 - ULA usada também para calcular endereço e incrementar o PC
 - Memória usada para armazenar instrução E dados
- A organização da parte operativa pode ser re-estruturada em função destas características
- Sinais de controle não serão determinados diretamente pela instrução, e sim pela instrução+etapa
- Usaremos uma máquina de estados finita para controle



Unidade Operativa Multiciclo

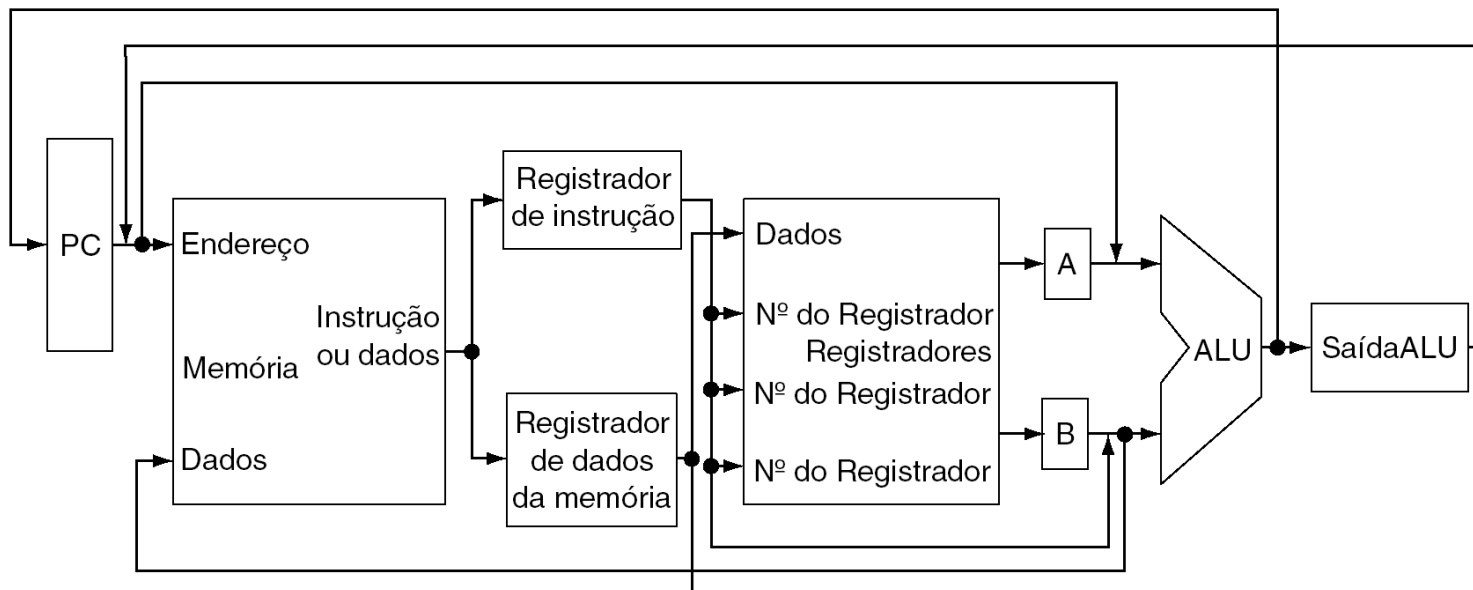
- Dividir as instruções em etapas, cada uma durando um ciclo
 - equilibrar a quantidade de trabalho a ser feito
 - restringir cada ciclo para usar apenas uma vez cada unidade funcional
- No final de um ciclo
 - armazenar os valores para uso em ciclos posteriores
 - lembrar circuitos sequenciais
 - Logo, necessita introduzir registradores auxiliares “internos” adicionais



Unidade Operativa Multiciclo

■ Visão Abstrata:

- Uma única ULA
- Uma única Unidade de Memória
- Registradores Auxiliares após unidades funcionais
 - onde colocá-los? Unidade combinacional de 1 ciclo e onde necessitar ser salvo. No nosso caso:



Obs.:

Dados utilizados pelas etapas subsequentes armazenados em registradores (elemento de estado invisível ao programador)

Dados utilizados pelas instruções subsequentes armazenados na Memória, Banco de Registradores (visível ao programador)



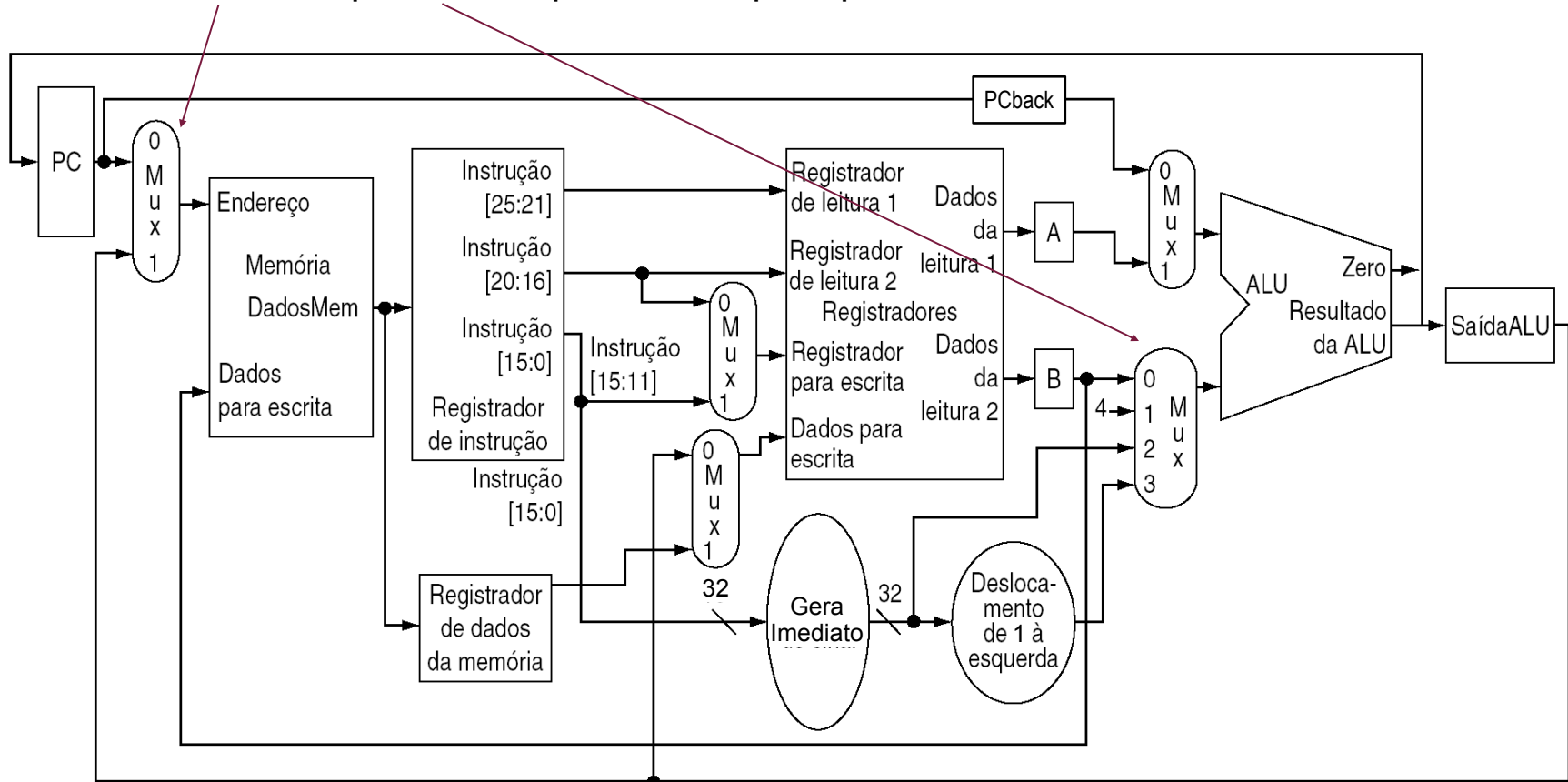
Unidade Operativa Multiciclo

■ Detalhamento:

- Todos os registradores auxiliares precisam manter o dado por 1 ciclo apenas (EXCEÇÃO IR)

Logo não necessitam de pino de controle de escrita

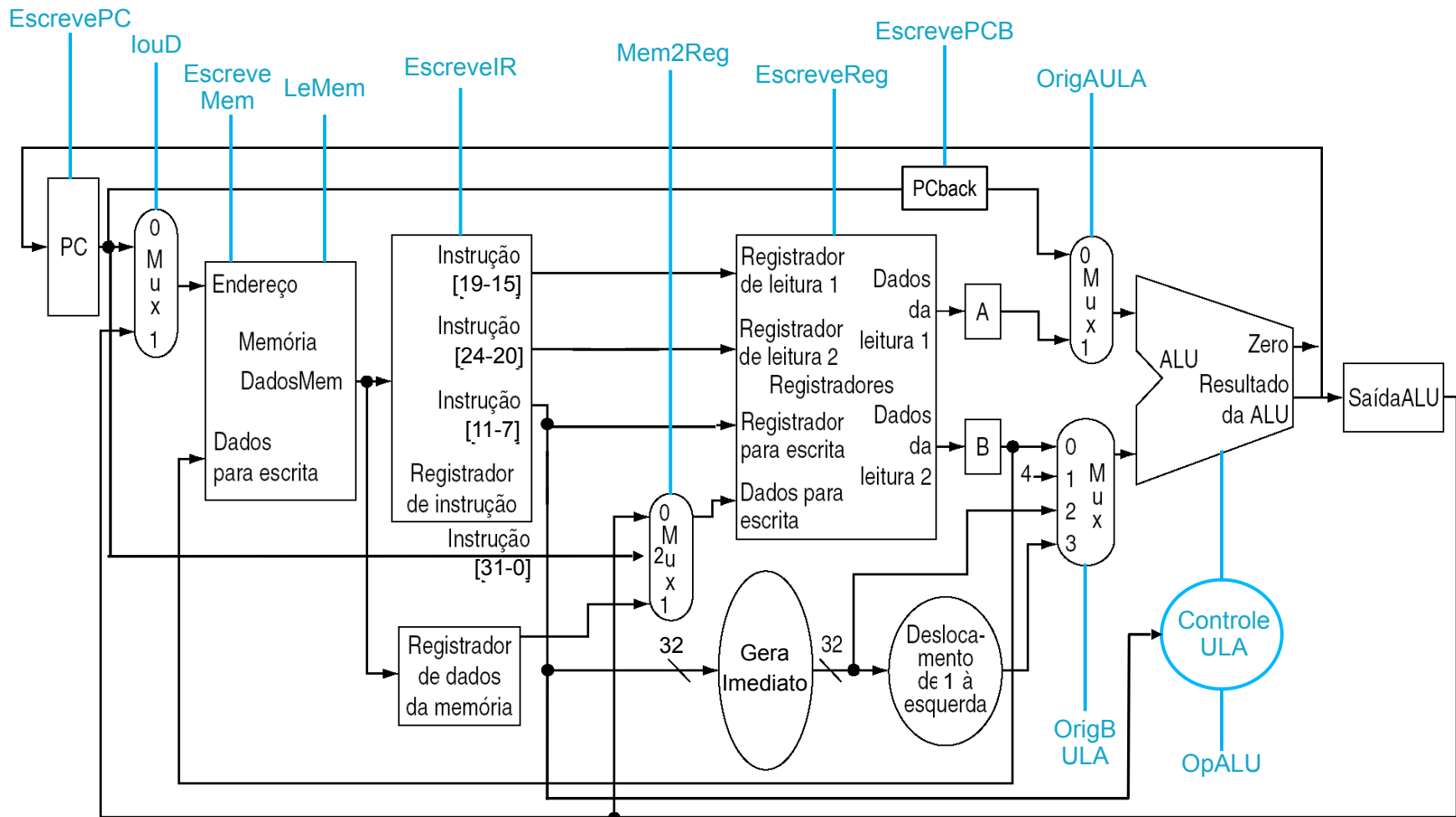
- Incluir e expandir Multiplexadores para poder reusar Unidades Funcionais





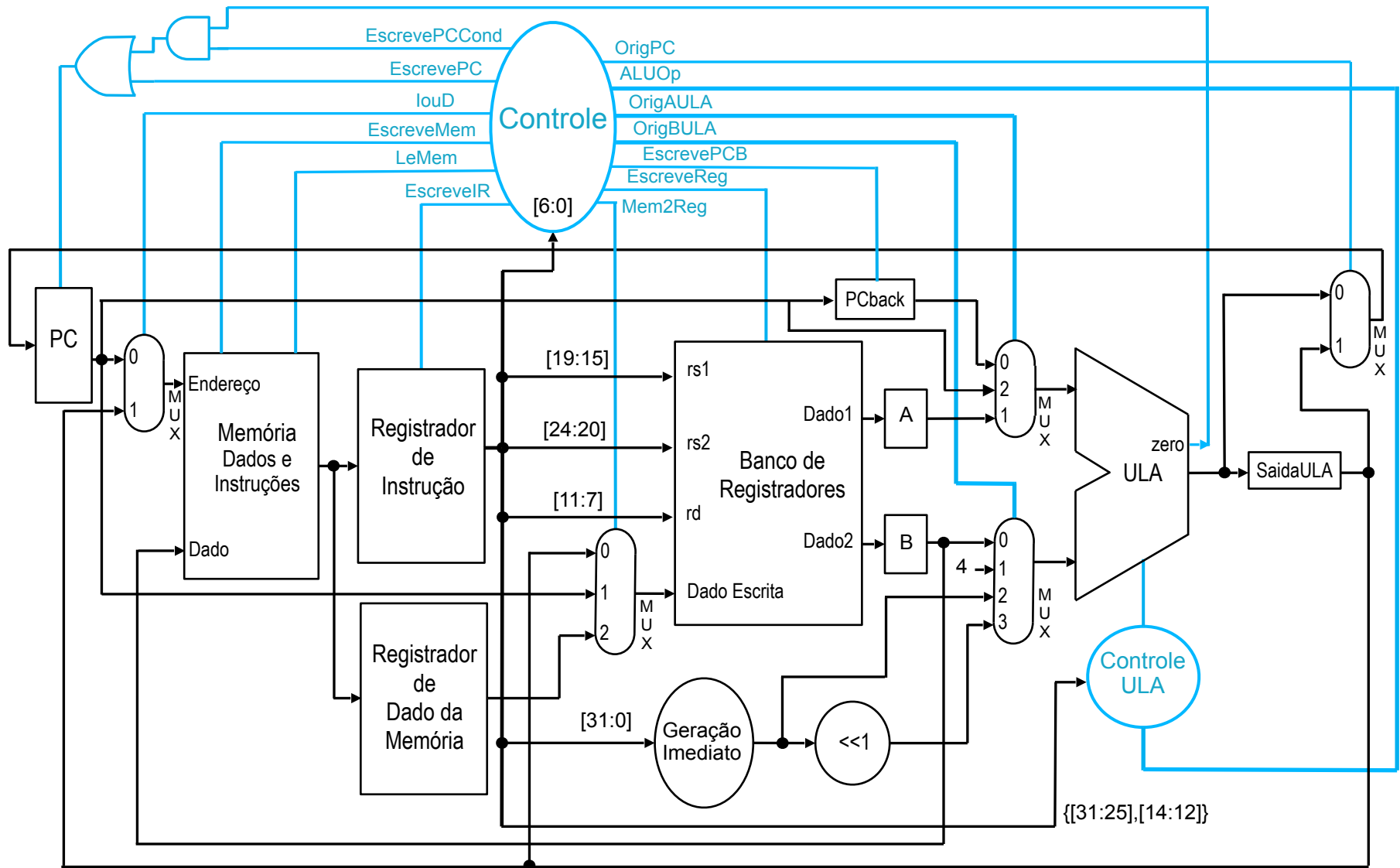
Caminho de Dados Multiciclo

Definição das Linhas de Controle





Caminho de Dados Multiciclo





Ações dos Sinais de Controle

Sinal	Valor	Efeito
EscreveReg	0	Não permite a escrita no Banco de Registradores
	1	O Registrador de Destino é escrito com o Dado de Escrita
OpALU	00	Soma
	01	Subtração
	10	Os campos {Funct7,Funct3} definem a operação
OrigAULA	0	Primeiro argumento da ULA é o registrador PC
	1	Primeiro argumento da ULA é o registrador A
OrigBULA	00	Segundo argumento da ULA é o registrador B
	01	Segundo argumento da ULA é o número 4
	10	Segundo argumento da ULA é o Imediato
	11	Segundo argumento da ULA é o Imediato deslocado 1 bit
LeMem	0	Desabilita a leitura da Memória
	1	Habilita a leitura da Memória
EscreveMem	0	Desabilita a escrita na Memória
	1	Habilita a escrita na Memória
Mem2Reg	00	O valor escrito no Banco de Registradores vem da SaídaALU
	01	O valor escrito no Banco de Registradores vem do PC
	10	O valor escrito no Banco de Registradores vem do MDR
IouD	0	O PC é usado para endereçar a memória
	1	O registrador SaídaALU é usado para endereçar a memória
EscreveIR	0	Desabilita a escrita no IR
	1	Habilita a escrita no IR
EscrevePC	0	Desabilita a escrita incondicional no PC
	1	Habilita a escrita incondicional no PC
EscrevePCCond	0	Desabilita a escrita condicional no PC
	1	Habilita a escrita condicional no PC
EscrevePCB	0	Desabilita a escrita do PCBack
	1	Habilita a escrita no PCBack
OrigPC	0	O valor escrito no PC é o resultado da ULA
	1	O valor escrito no PC é o registrador SaídaULA



Controle Multiciclo

■ Exemplo: Instrução Tipo-R

poderia ser pensada na seguinte instrução em linguagem de descrição de hardware (HDL):

$$\text{Reg}[\text{Memory}[\text{PC}][15:11]] \leq \text{Reg}[\text{Memory}[\text{PC}][25:21]] \text{ op } \text{Reg}[\text{Memory}[\text{PC}][20:16]] ;$$

Decomposta nas seguintes etapas usando “variáveis locais”:

$\text{IR} \leq \text{Memory}[\text{PC}] ;$	— busca instrução
$\text{A} \leq \text{X}[\text{IR}[25:21]] ;$	— busca operandos
$\text{B} \leq \text{X}[\text{IR}[20:16]] ;$	
$\text{saidaALU} \leq \text{A op B} ;$	— execução operação
$\text{X}[\text{IR}[15:11]] = \text{saidaALU} ;$	— escreve resultado
$\text{PC} \leq \text{PC} + 4 ;$	— calcula endereço próxima instrução



Controle Multiciclo: 5 Etapas

Considerando:

- 1 acesso à memória, 1 acesso ao banco de registradores ou 1 operação da ULA por ciclo. (determina o ciclo de clock)
- Projeto acionado por transição (valores dos regs não muda até então)

1. Etapa de Busca da Instrução

IR $\leq$ *Memoria*[*PC*];

PCback $\leq$ *PC*;

PC $\leq$ *PC* + 4;

Execução: Ler a Instrução da memória e incrementar PC

- Ativar os sinais de controle LeMem e Escreve IR
- Definir PC como origem do endereço: Colocar *loutD* em 0
- Incrementar PC+4: OrigAALU em 0 (PC para ALU), OrigBALU 01 (4 para ALU) e opALU 00 (soma).
- Armazenar o endereço de instrução incrementado em PC: origPC 00 e ativar EscrevePC
- Armazenar PC atual em PCback: ativar EscrevePCB



Controle Multiciclo...

2. Etapa de Decodificação da Instrução e busca de operandos

$A \leq X[IR[25:21]];$

$B \leq X[IR[20:16]];$

$SaidaALU \leq PCback + (imediato \ll 1);$

Execução: Como não sabemos qual a instrução ainda, podemos fazer ações gerais que não causem problemas depois!

- Acessar o banco de registradores e ler rs1 e rs2 e armazenar em A e B.
- Cálculo prévio do endereço de desvio e coloca em SaidaALU: OrigAALU em 0 (PCback para ALU), OrigBALU em 11 (imediato $\ll 1$ para ALU) e opALU 00 (soma).



3. Etapa de Execução, cálculo do endereço de memória ou conclusão de desvio

Referência à memória (lw,sw):

$$SaidaULA = A + imediato$$

Execução: Calcular o endereço

- OrigAALU em 1 (reg A), OrigBALU em 10 (imediato), opALU 00 (soma)

Instrução Tipo-R:

$$SaidaULA \leq A \text{ op } B$$

Execução: os valores de A e B estão prontos!

- OrigAALU em 1 (A na ALU), OrigBALU em 00 (B na ALU), opALU 10 (lógico-aritmética pelo campo funct7 e funct3)

beq:

$$\text{if } (A == B) \text{ PC} \leq SaidaALU$$

Execução: o endereço de desvio já está pronto.

- OrigAALU em 1 (A na ALU), OrigBALU em 00 (B na ALU), opALU 01 (subtrai)
- Ativar sinal EscrevePCCond
- Colocar OrigPC em 01 (valor do PC vem da ULA)

Jal:

$$\text{PC} \leq SaidaULA; X[rd] \leq \text{PC} + 4 \text{ (já no PC)}$$

Execução: PC é substituído pelo endereço de salto, salva endereço de retorno PC+4

- Colocar OriPC em 1 (saída da ULA)
- Ativar EscrevePC e EscreveReg
- Mem2Reg em 01 (PC)



Controle Multiciclo ...

4. Etapa de acesso à memória ou conclusão de instrução Tipo-R

Referência à memória (lw, sw):

$MDR \leq Memória[SaidaALU];$ #para load

ou

$Memória[saidaALU] \leq B$ # para store

Execução: Em qualquer dos casos o endereço já está na SaidaALU

- loutD colocado em 1 (Endereço da ALU)
- LeMem ou EscreveMem deve ser ativado

Instrução Tipo-R:

$X[IR[15:11]] \leq SaidaALU$

Execução: Colocar o resultado no reg destino

- Ativar EscreveReg
- Colocar Mem2Reg em 0 (saída da ALU na entrada do banco de reg)



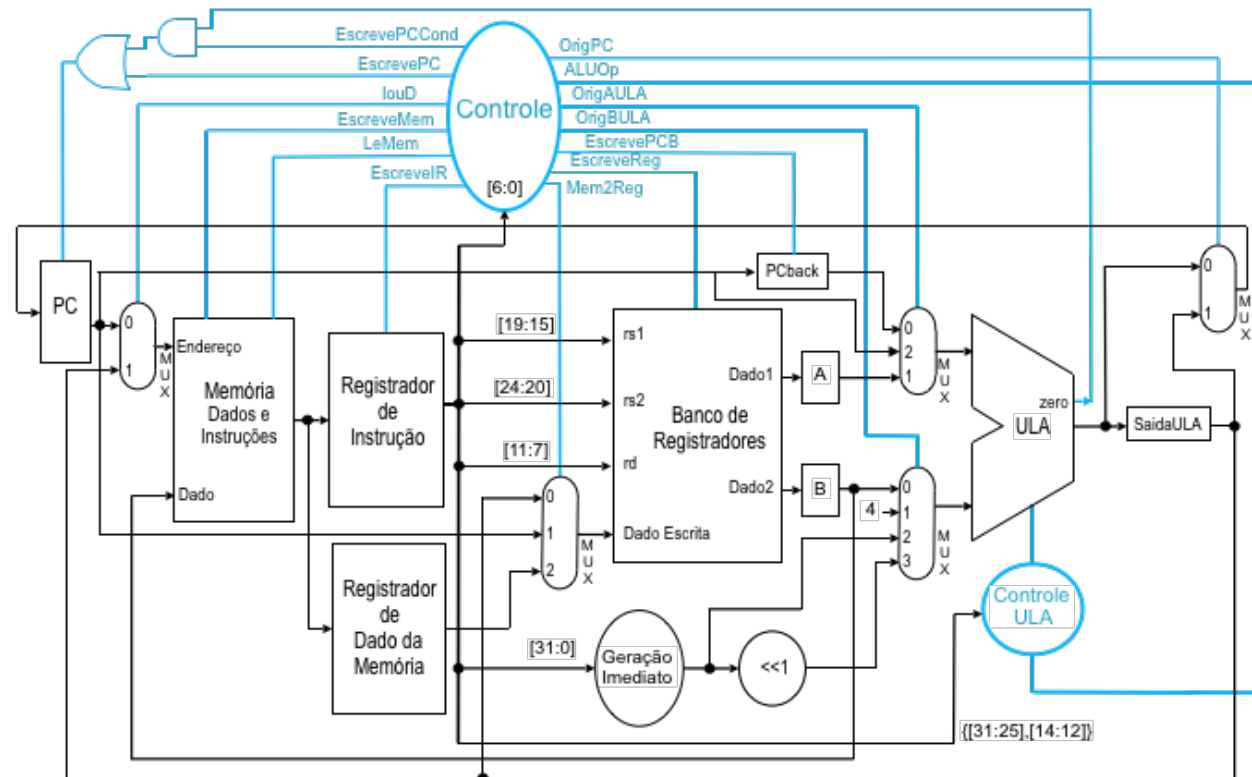
Controle Multiciclo ...

5. Conclusão da Leitura da Memória (lw)

$X[rd] \leftarrow MDR;$

Execução: Escrever o dado do MDR no banco de reg

- Colocar MemparaReg em 1 (resultado da memória)
- Ativar o EscreveReg





Resumo Controle Multiciclo

Etapa	Tipo-R	Acesso à Memória	Desvios Condicionais	Desvios Incondicionais
Busca da Instrução	$IR \leq Mem[PC]$ $PC_{back} \leq PC$ $PC \leq PC+4$			
Decodificação, Leitura dos registradores	$A \leq X[IR[19:15]]$ $B \leq X[IR[24:20]]$ $SaidaULA \leq PC + imm \ll 1$			
Execução, cálculo do endereço	$SaidaULA \leq A \text{ op } B$	$SaidaULA \leq A + imm$	Se $(A == B)$ $PC \leq SaidaULA$	$Reg[IR[11:7]] \leq PC + 4$ $PC \leq SaidaULA$
Acesso à memória, conclusão tipo-R	$X[IR[11:7]] \leq SaidaULA$	Load: $MDR \leq Mem[SaidaULA]$ Store: $Mem[SaidaULA] \leq B$		
Conclusão lw		Load: $X[IR[11:7]] \leq MDR$		

Ex.: Considerando um workload de 25% loads, 10% stores, 11% branches, 2% jumps e 52% operações com ALU. Qual a CPI média implementada?

$$CPI = 0.25 \times 5 + 0.1 \times 4 + 0.52 \times 4 + 0.11 \times 3 + 0.02 \times 3 = 4.12$$

Lembrando que no pior caso: $CPI = 5$!